Trust-Weighted Gossip for Decentralized Storage and Retrieval

A Protocol for Returning Information Custody to the Social Graph

Version 1.0

Leon Acosta
leon@nostr-wot.com

March 2026

Abstract

Decentralized social protocols—Nostr, ActivityPub (Mastodon), AT Protocol (Bluesky)—still depend on servers that control what gets stored, served, and censored. This paper presents an open, censorship-resistant protocol for social media and messaging that returns data custody to the social graph itself. The protocol inherits Nostr's proven primitives—secp256k1 identity, signed events, relay transport—and adds a storage and retrieval layer where users own their data. Because events are self-authenticating (signed by the author's keys), they are portable across protocols: convertible to and from ActivityPub, AT Protocol, and other decentralized formats. Users form reciprocal *storage pacts* with trust-weighted peers, creating a distributed storage mesh that mirrors how human communities naturally preserve and transmit information. All protocol intelligence—gossip forwarding, blinded matching, WoT filtering, device resolution—lives in clients. Standard Nostr relays work without any modifications; optimized relays can optionally accelerate specific operations. We describe the pact formation mechanism, a tiered retrieval protocol with cascading fallback paths, and a Web of Trust (WoT)-filtered gossip layer that bounds propagation while maintaining epidemic delivery guarantees. We further describe how integration with FIPS (Free Internetworking Peering System) extends the protocol to operate across heterogeneous transports including mesh radio, Bluetooth, and overlay networks, eliminating dependence on the internet itself.

1 Introduction

1.1 The Gossip Parallel

Human communities have always propagated information through gossip. A person shares news with their close circle, who share it with theirs, creating epidemic spread through a trust-weighted social graph. This mechanism has three properties that formal gossip protocols seek to replicate:

1. **Trust filtering**—information from a close friend carries more weight than from a stranger. A claim about person X means different things coming from someone 1 hop versus 4 hops away.
2. **Contextual preservation**—gossip within a community preserves context: who said what, to whom, under what circumstances. Gossip without context is slander; with context it is signal.
3. **Natural redundancy**—important information reaches you through multiple independent paths. If one friend is unavailable, another provides the same update. The redun-

dancy is proportional to the information's social relevance.

These properties map directly onto the protocol primitives we describe: WoT-filtered forwarding (Section 4), volume-matched storage pacts (Section 5), and multi-path retrieval with cascading fallback (Section 6).

1.2 The Relay Problem

Decentralized social protocols promise user sovereignty, but all of them depend on servers that control what gets stored, served, and censored. In Nostr [1], relays decide which events to keep and for how long. In Mastodon/ActivityPub [12], your account is bound to an instance operator who can suspend it, shut down, or change their terms. In Bluesky's AT Protocol, a centralized relay aggregates and serves the firehose. The specific architecture varies, but the dependency is the same:

- **Storage control**—servers decide which data to store and for how long.
- **Distribution control**—servers decide which data to serve and to whom.
- **Censorship capability**—operators can silently drop content or suspend accounts.
- **Economic leverage**—users must pay operators or accept their terms.

This recreates the platform dependency that decentralized protocols were designed to eliminate. The user's social graph lives on infrastructure they do not control.

1.3 Contribution

We propose a local-first architecture where:

- **Primary storage** resides on the user's device and their close WoT peers (1–2 hops).
- **Primary retrieval** queries the trust network first, not relay infrastructure.
- **Relay role** is demoted to optional discovery and curation—useful for beyond-graph reach, not required for existence.
- **Data is self-authenticating and portable.** Every event is signed by the author's keys. Data can be converted to and from ActivityPub, AT Protocol, and other decentralized formats—users own their data, not the protocol.

This is gossip protocol in the computer science sense—epidemic information spreading through a peer mesh [2]—with WoT determining propagation priority. The protocol inherits Nostr's proven primitives—secp256k1 keypairs, signed events, relay transport—and works with existing Nostr relays and clients from day one. All protocol intelligence lives in clients; standard relays store and forward events without modification. The combination of Nostr's key model, social-graph storage, and trust-weighted routing creates a system where the network's social structure *is* its infrastructure.

The paper makes three contributions:

1. A **storage pact mechanism** with volume-balanced reciprocity, challenge-response verification, and popularity-scaled redundancy (Section 5).
2. A **tiered retrieval protocol** with four delivery paths—local, cached endpoint, gossip, and relay fallback (Section 6).
3. A **FIPS integration architecture** that extends the protocol to operate over mesh radio, Bluetooth, serial links, and other transports without internet dependence (Section 8).

2 Communities and Protocols: A Structural Parallel

The design of this protocol is not merely inspired by human social dynamics—it is structurally isomorphic to them. Each protocol primitive maps to an observable pattern in how humans form communities, maintain relationships, and propagate information.

2.1 Inner Circle, Outer Circle, Acquaintances

Human social networks exhibit concentric structure. Robin Dunbar's research [3] identifies layers: ∼5 intimate contacts, ∼15 close friends, ∼50 good friends, ∼150 casual friends, with each layer roughly tripling in size and halving in intimacy.

The protocol mirrors this with three node roles:

Human Layer	**Protocol Equivalent**	**Persona**	**Storage Obligation**	**Uptime**
Inner circle (5–15)	Full nodes (25%)	*Keeper*	Complete event history	95%
Extended circle (50–150)	Light nodes (75%)	*Witness*	Rolling 30-day window	30%
Acquaintances (150+)	Relay-discovered peers	*Herald*	None (optional caching)	Variable

Full nodes are the protocol's equivalent of the friends who remember everything—your complete history is safe with them. Light nodes are the broader social circle: they know what you've been up to recently, but don't maintain archives. Acquaintances discovered through relays are the weak ties [4] that provide reach beyond your community, without storage commitment.

2.2 Reciprocity as Infrastructure

In human communities, relationships survive through reciprocity. You remember my stories; I remember yours. One-sided relationships decay. The protocol formalizes this as *storage pacts*: bilateral agreements where both parties store each other's events, matched by data volume within a 30% tolerance.

This volume matching is the protocol equivalent of activity-matched friendships. A prolific poster (high volume) paired with a lurker (low volume) creates an asymmetric obligation that incentivizes defection—just as asymmetric friendships decay in real social networks. The protocol prevents this by matching peers with compatible activity levels.

2.3 Reputation Through Behavior, Not Scores

Human reputation is not a number. It is the aggregate of observed behavior, weighted by the observer's trust in each source. The protocol's reliability scoring operates identically:

- Each node maintains **per-peer reliability scores** using a 30-day rolling window of challenge-response results.
- Scores are private—each node computes its own assessment.
- A score above 90% is healthy; between 50–70% triggers replacement; below 50% means immediate drop.
- Failed peers naturally lose their reciprocal storage—the same way unreliable friends are gradually excluded from information flow.

There is no global reputation score. The network topology *is* the incentive structure. A node with 20 reliable pact partners has 20 advocates forwarding its content through gossip. A dropped pact means a lost advocate and reduced reach—the protocol equivalent of being talked about less.

2.4 Guardianship—Bootstrapping Through Generosity

Every community has patrons—established members who vouch for newcomers they don't personally know. A shopkeeper who gives a stranger their first job. A neighbor who co-signs a lease for a new arrival. These acts of generosity bootstrap trust where none exists.

The protocol formalizes this as *guardian pacts*: an established user (*Guardian*) voluntarily stores data for one untrusted newcomer (*Seedling*) outside their Web of Trust. Unlike bootstrap pacts (triggered by a follow), guardian pacts are volunteered—the Guardian opts in, accepting a small storage cost to help the network grow.

	Bootstrap Pact	**Guardian Pact**
Trigger	Seedling follows someone	Guardian volunteers
WoT required	No (follow creates edge)	No
Capacity	One per followed user	One per Guardian
Expiry	90 days or 10 reciprocal pacts	90 days or Hybrid phase (5+ pacts)
Reciprocity	One-sided	One-sided

The pay-it-forward framing is deliberate: today's Seedling becomes tomorrow's Guardian. A user who was supported during their bootstrap phase is encouraged (by client UX, not protocol enforcement) to volunteer a guardian slot once they reach Sovereign phase.

2.5 Gossip as Curated Propagation

When humans gossip, they don't broadcast to everyone. They share selectively based on trust and relevance. The protocol's WoT-filtered forwarding mirrors this:

- **Active pact partners** (highest priority)—you forward eagerly for those whose data you store.
- **1-hop WoT peers**—people you directly follow. You'll pass along their content.
- **2-hop WoT peers**—friends-of-friends. You'll forward if capacity permits.
- **Unknown sources**—never forwarded. Strangers' gossip stops at your door.

This creates a natural boundary for information propagation that matches Dunbar's social brain hypothesis [3]: the protocol's gossip radius is bounded by the same trust gradient that bounds human social information flow.

3 Network-Theoretic Foundations

The protocol's design is grounded in results from network science. This section maps each key design decision to the formal theory that justifies it, establishing why the social graph is a viable substrate for decentralized infrastructure.

3.1 Small-World Property and Gossip Reach

Watts and Strogatz [13] demonstrated that networks with high clustering and short path lengths—*small-world* networks—emerge from even minor random rewiring of regular lattices.

In the small-world regime, characteristic path length scales as $L \sim \ln(N)/\ln(k)$ while the clustering coefficient C remains close to its lattice value ($\sim 3/4$). Social networks consistently exhibit this structure: densely clustered local neighborhoods connected by a few long-range shortcuts.

This directly justifies the protocol's gossip TTL of 3 hops. In a small-world network of N nodes with mean degree k, the expected number of nodes reachable within h hops (accounting for clustering coefficient C) is:

$$\text{reach}(h) \approx k \cdot [k(1-C)]^{h-1} \tag{1}$$

For a 5,000-node network with $k = 20$ and $C = 0.25$: reach(1) = 20, reach(2) = 300, reach(3) $\approx$ 4,500. Three hops cover 90%+ of the network. Increasing TTL to 4 would add minimal reach at substantial bandwidth cost—the logarithmic path length means most information arrives within 2–3 hops.

The small-world structure also explains why WoT-bounded gossip works at all: the combination of local clustering (friends share friends) and short paths (any two nodes are connected by a few hops) means that a gossip message restricted to 2-hop WoT peers can still reach most of the relevant network through trust-weighted forwarding.

3.2 Scale-Free Topology: Robustness and Vulnerability

The protocol's simulation uses Barabási–Albert (BA) preferential attachment [14] to generate social graphs with power-law degree distributions $P(k) \sim k^{-\gamma}$. This choice is not arbitrary: empirical studies consistently find scale-free structure in online social networks, with γ typically between 2 and 3.

Albert, Jeong, and Barabási [15] proved a fundamental asymmetry in scale-free networks:

- **Random failure**: Scale-free networks tolerate the random removal of up to 80–95% of nodes before losing their giant component. Low-degree nodes (the vast majority) contribute little to global connectivity, so their failure has negligible effect.
- **Targeted attack**: Removing nodes in order of decreasing degree fragments scale-free networks after removing as few as 5–18% of the highest-degree hubs.

Cohen et al. [16] formalized this using percolation theory. The giant component survives random removal as long as $\kappa = \langle k^2 \rangle / \langle k \rangle > 2$ (the Molloy–Reed criterion). For scale-free networks with $\gamma \leq 3$, the second moment $\langle k^2 \rangle$ diverges, so $\kappa \to \infty$ and the percolation threshold approaches 1—the network is effectively immune to random failure.

These results directly inform three protocol design decisions:

1. **Pact redundancy (20 active + 3 standby)**: Random node failures (mobile devices going offline, churn) are the dominant failure mode. With 23 pact partners, the protocol exploits scale-free robustness—data survives because the loss of any individual partner is statistically insignificant.
2. **Eclipse attack defense**: The targeted-attack vulnerability motivates the WoT-only pact formation rule. An attacker must first infiltrate the target's WoT (mutual follows) before offering pacts—a social barrier that prevents the degree-based targeting that would be devastating in an open network.
3. **Standby pact promotion**: The 3 standby pacts provide immediate failover without renegotiation. This addresses the narrow window between targeted hub removal and network fragmentation: if a high-degree node's pact partners are taken down, standby promotion maintains connectivity during the recovery period.

3.3 Epidemic Spreading and the WoT Boundary

Pastor-Satorras and Vespignani [17] proved that the epidemic threshold for spreading processes on scale-free networks vanishes:

$$\lambda_c = \frac{\langle k \rangle}{\langle k^2 \rangle} \to 0 \quad (\text{for } \gamma \leq 3) \tag{2}$$

This means any non-zero transmission rate produces epidemic spreading across the entire network. For gossip protocols, this is a double-edged result: gossip *will* spread efficiently (good for retrieval), but so will spam, misinformation, and attack traffic (bad for security).

The protocol resolves this tension by restricting gossip propagation to the WoT boundary (2-hop maximum). Castellano and Pastor-Satorras [18] showed that the epidemic threshold on a bounded-degree subgraph is governed by the spectral radius λ_1 of the adjacency matrix:

$$\lambda_c = \frac{1}{\lambda_1} \geq \frac{1}{\sqrt{k_{\max}}} \tag{3}$$

When gossip is confined to a 2-hop WoT neighborhood, the effective maximum degree $k_{\max}$ is bounded by Dunbar-layer sizes ($\sim$150), giving $\lambda_c \geq 1/\sqrt{150} \approx 0.08$—a finite, non-trivial threshold. Content that falls below this spreading rate dies out locally rather than propagating globally.

This creates a **dual regime**: within the WoT boundary, gossip spreads efficiently (approaching the scale-free guarantee); beyond the boundary, propagation requires relay assistance. The protocol's four-tier retrieval cascade (Section 6) is the operational implementation of this theoretical boundary: Tiers 1–3 exploit intra-WoT epidemic spreading, while Tier 4 (relay fallback) handles the inter-community gap.

3.4 Triadic Closure and 2-Hop WoT Effectiveness

Rapoport [19] first formalized triadic closure: if A is connected to both B and C, the probability that B and C become connected is significantly higher than random. This structural tendency drives the clustering coefficient in social networks and is the mechanism that makes 2-hop WoT meaningful.

The protocol's 2-hop tier exploits triadic closure directly. If Alice (node A) mutually follows Bob (node B), and Bob follows Carol (node C), then:

- The probability that Carol is relevant to Alice is proportional to the number of Alice's mutual contacts who follow Carol (the trust score).
- A Carol followed by 8 of Alice's 15 mutual contacts is far more likely to be a genuine community member than one followed by 1.

The trust score in the 2-hop WoT map is precisely this count of closing triads. Granovetter's weak ties theory [4] complements this: the 1-hop tier (non-mutual follows) represents weak ties that bridge communities, while the direct WoT tier (mutual follows) represents strong ties with high triadic closure. The 50/30/20 weight split reflects Granovetter's finding that strong ties provide redundancy (high overlap in information) while weak ties provide novelty (access to distant communities).

3.5 Community Structure and Information Boundaries

Girvan and Newman [20] demonstrated that real networks have modular community structure detectable through edge betweenness. Edges connecting communities carry disproportionately many shortest paths—they are information bottlenecks.

The modularity metric Q quantifies this structure:

$$Q = \sum_c \left[\frac{L_c}{m} - \left(\frac{d_c}{2m} \right)^2 \right] \tag{4}$$

where L_c is edges within community c, m is total edges, and d_c is the total degree of community c. Values of $Q > 0.3$ indicate significant community structure.

This has two implications for the protocol:

1. **Gossip confinement**: In high-modularity networks (typical of social graphs), gossip propagating within a WoT community rarely crosses community boundaries because few edges span the gap. This is a *feature*: read requests for a community member's content are served by community members, keeping traffic local. The inter-community relay fallback handles the exceptions.

2. **Privacy through topology**: The blinded request mechanism (Section 6) provides cryptographic privacy, but community structure provides *topological* privacy as an additional layer. A gossip request for Alice's data propagates through Alice's community—nodes outside the community never see the request, regardless of blinding.

3.6 Redundant Paths and Fault Tolerance

Menger's theorem [21] states that the maximum number of vertex-disjoint paths between two nodes equals the minimum vertex cut separating them. For a k-vertex-connected graph, any node can tolerate the failure of $k - 1$ neighbors without losing connectivity to any other node.

Applied to the pact topology: if a user maintains 20 active pact partners, and each partner is connected to the user through the WoT, then Menger's theorem guarantees that an adversary must compromise at least as many nodes as the minimum vertex cut to isolate the user's data. In practice, the WoT's high clustering means multiple independent paths exist between pact partners, providing redundancy beyond what the raw pact count suggests.

The standby pact mechanism adds a layer: even after min-cut nodes are compromised, the 3 standby partners (selected for path diversity) provide fallback paths that may traverse different WoT communities entirely.

3.7 Dunbar Layers as Protocol Parameters

Dunbar's research [3] identifies fractal social layers: ∼5 intimate contacts, ∼15 close friends, ∼50 good friends, ∼150 casual friends, each layer roughly 3× the previous. The protocol's parameters are calibrated to these layers:

Dunbar Layer	Size	Protocol Mapping	Parameter
Support clique	∼5	Full-node pact partners	∼5 Full pacts (25% of 20)
Sympathy group	∼15	Active pact partners	20 active pacts
Affinity group	∼50	Direct WoT tier	Mutual follows
Dunbar number	∼150	1-hop + 2-hop WoT tiers	Follow list + extended network
Acquaintances	∼500	Relay-discovered peers	Beyond WoT boundary

The 20-pact target sits within the sympathy group layer (∼15), reflecting that storage pacts require ongoing reciprocal obligation—a level of commitment compatible with the ∼15 relationships humans maintain with regular contact. The 2-hop WoT boundary (∼150 effective peers) aligns with Dunbar's number, beyond which trust assessment becomes unreliable.

This is not coincidence. The protocol's viability depends on the social graph's structure, and that structure is constrained by human cognitive limits. A protocol that required 500 pact partners would exceed the sympathy group's capacity; one that relied on 5-hop WoT would extend trust beyond the Dunbar boundary. The parameter choices respect the cognitive architecture that generates the social graph.

4 Protocol Primitives

4.1 Identity and Trust

Each participant holds a secp256k1 keypair, compatible with Nostr's identity model [1]. The public key serves as both identity and address. A key hierarchy isolates device-level operations from identity-level authority:

- **Root key** (cold storage)—signs device delegations only.
- **Governance key**—signs profile and follow list updates.
- **Device subkeys**—sign day-to-day events (posts, reactions, DMs).

The Web of Trust is defined by follow relationships. A node p follows a set of nodes F_p. The WoT distance $d(p, q)$ is the minimum number of follow-hops from p to q. The protocol operates within a 2-hop boundary: nodes forward gossip only to peers where $d(p, q) \leq 2$.

4.2 Node Types

Let $N = \{n_1, n_2, \ldots, n_k\}$ be the set of network participants. Each node n_i has type $t_i \in \{\text{Full}, \text{Light}\}$:

- **Full nodes** (*Keepers*) maintain complete event history for their pact partners. Expected uptime: $u_{\text{full}} = 0.95$.
- **Light nodes** (*Witnesses*) maintain events within a checkpoint window W (default 30 days) for their pact partners. Expected uptime: $u_{\text{light}} = 0.30$.

The network composition follows the distribution observed in real deployments: approximately 25% Full nodes (always-on servers, dedicated hardware) and 75% Light nodes (mobile devices, intermittent connectivity).

4.3 Events

An event $e = (\text{id}, \text{author}, \text{kind}, \text{size}, \text{seq}, \text{prev_hash}, \text{created_at})$ is the atomic unit of information. Events are cryptographically signed by the author's device key and are immutable once published.

The weighted average event size under the default content mix is:

Formula F-01:

$$\mathbb{E}[\text{size}] = \sum_k \text{size}_k \cdot \text{mix}_k = 800(0.40) + 500(0.30) + 600(0.15) + 900(0.10) + 5500(0.05) = 925 \text{ bytes} \tag{5}$$

4.4 Checkpoints

Checkpoints (kind 10051) are periodic reconciliation markers published by each user. A checkpoint contains:

- Per-device event heads (latest event ID and sequence number per device).
- A Merkle root over all events since the previous checkpoint.
- References to current profile and follow list.

Checkpoints enable light nodes to verify data completeness without downloading full history, and define the storage obligation boundary for light pact partners.

5 Storage Pact Mechanism

5.1 Pact Formation

A storage pact is a bilateral agreement between two nodes to store each other's events. Formation follows a three-phase DVM-style (Data Vending Machine) negotiation:

1. **Request** (kind 10055): Node p broadcasts a storage pact request with its volume estimate, minimum pact count, and TTL.
2. **Offer** (kind 10056): Qualifying WoT peers respond with offers.
3. **Accept** (kind 10053): Both parties exchange private pact events and begin mutual storage.

Qualification requires: WoT membership (follows or followed-by), volume balance within tolerance δ, and minimum account age (default 7 days to prevent Sybil pact formation).

5.2 Volume Balancing

Let V_p and V_q be the data volumes of nodes p and q. A pact is balanced when:

$$\frac{|V_p - V_q|}{\max(V_p, V_q)} \leq \delta \tag{6}$$

where $\delta = 0.30$ (30% tolerance). This ensures symmetric risk: neither partner bears disproportionate storage cost.

5.3 Pact Topology

Each node maintains two classes of pact partners:

- **Active pacts** (default: 20): Partners that are regularly challenged and expected to serve data on request.
- **Standby pacts** (default: 3): Partners that receive events but are not challenged. Standby partners are promoted to active when an active partner fails, providing instant failover without renegotiation delay.

For high-follower accounts, the pact count scales:

Followers	**Active Pacts**
< 100	10
100–1,000	20
1,000–10,000	30
10,000+	40+

5.4 Proof of Storage

Pact partners are verified through periodic challenge-response exchanges (kind 10054):

Hash challenge: The challenger specifies a range of event sequence numbers and a nonce. The partner computes $H(\text{events}[i..j] \| \text{nonce})$ and returns the hash. This proves possession without transferring full events.

Serve challenge: The challenger requests a specific event by sequence number and measures response latency. Consistently slow responses (>500ms) suggest the partner is proxying rather than storing locally. Flagged peers receive 3× challenge frequency.

5.5 Reliability Scoring

Each node maintains per-peer reliability scores using an exponential moving average:

$$\text{score}' = \text{score} \cdot \alpha + \text{result} \cdot (1 - \alpha) \tag{7}$$

where $\alpha = 0.95$ and result $\in \{0, 1\}$. This gives a 30-day effective window with recent challenges weighted more heavily.

Score	Status	Action
$\geq 90\%$	Healthy	No action
70–90%	Degraded	Increase challenge frequency
50–70%	Unreliable	Begin replacement negotiation
$< 50\%$	Failed	Drop immediately, promote standby

5.6 Storage Obligations

The total storage obligation per user depends on the pact partner composition:

Formula F-03 (storage per user):

$$S = P \cdot (f \cdot \mathbb{E}[\text{size}] \cdot R \cdot D_{\text{full}} + (1 - f) \cdot \mathbb{E}[\text{size}] \cdot R \cdot D_{\text{light}}) \tag{8}$$

where P = number of active pacts (default 20), f = fraction of Full node partners ($\sim$0.25), $\mathbb{E}[\text{size}] = 925$ bytes, R = events per day (default 25), D_{full} = complete history duration, $D_{\text{light}} = \min(\text{duration}, W)$.

5.7 Guardian Pacts

Guardian pacts extend the pact mechanism to support newcomers who have no Web of Trust presence. An established user (a *Guardian*) volunteers to store data for one newcomer (a *Seedling*) without WoT membership or volume matching requirements.

Guardian pacts use the same kind 10053 event with a `type: guardian` tag. Formation flow:

1. **Opt-in**: A Sovereign-phase node advertises guardian availability via kind 10055 with `type: guardian`.
2. **Match**: A Seedling with fewer than 5 pacts is matched (client-side or relay-assisted).
3. **Accept**: Both parties exchange kind 10053 with `type: guardian`.
4. **Store**: The Guardian stores the Seedling's events. Challenge-response verification (kind 10054) applies.

Expiry conditions: 90 days from formation, or the Seedling reaches Hybrid phase (5+ reciprocal pacts). Each Guardian holds at most one active guardian pact, bounding the storage cost to a single user's data volume.

6 Retrieval Protocol

6.1 Delivery Tiers

When a node needs to retrieve events from a followed author, the protocol attempts delivery through a cascade of increasingly expensive paths:

Tier 0—BLE mesh (nearby): Nearby devices serve events via Bluetooth Low Energy mesh, relayed up to 7 hops. No internet required. Interoperable with FIPS transport layer (Section 8). Latency: variable.

Tier 1—Instant (local): The node already stores the author's events locally, either through an active pact or from a previous read cache. Cost: zero network traffic. Latency: zero.

Tier 2—Cached Endpoint: The node has cached endpoint addresses (kind 10059) for the author's storage peers. A direct connection retrieves the events without gossip overhead. Latency: ∼60ms base + 20ms jitter.

Tier 3—Gossip (kind 10057): The node broadcasts a blinded data request to its WoT peers with TTL=3. The request uses a blinded pubkey $bp = H(\text{target_pubkey} \| \text{YYYY-MM-DD})$ that rotates daily. Latency: ∼80ms per hop + 30ms jitter per hop.

Tier 4—Relay Fallback: After a configurable timeout (default 30s), the node falls back to a traditional relay query. This is the path of last resort. Latency: ∼200ms base + 50ms jitter.

Each successive tier is attempted only when the previous tier fails or times out, creating a natural cost gradient that keeps most traffic within the social graph.

6.2 Read Cache and Cascading Replication

A critical property emerges from the retrieval protocol: **reads create replicas**. When Bob fetches Alice's events from a storage peer, Bob now holds a local copy. When Carol subsequently requests Alice's events via gossip, Bob can respond—without being one of Alice's formal pact partners.

This creates cascading read-caches that replicate popular content across the follower base:

1. Alice's 20 pact partners hold her events.
2. Each of Alice's followers who reads her events becomes an informal cache.
3. Subsequent readers find the data closer in the social graph.
4. Load scales with $O(\text{followers})$, not $O(\text{pact_partners})$.

The read cache is bounded (default 100MB) and LRU-evicted.

6.3 Blinded Requests and Privacy

Data requests (kind 10057) use blinded pubkeys to preserve reader privacy:

$$bp = H(\text{target_root_pubkey} \| \text{YYYY-MM-DD}) \tag{9}$$

The blinding salt rotates daily. Peers match incoming requests against both today's and yesterday's salts to handle clock skew. This ensures:

- Storage peers learn that *someone* requested the data, but not *who*.
- Observers cannot link requests across days.
- The requesting node's reading patterns are not exposed to the gossip network.

6.4 Gossip Reach Analysis

With a mean degree of 20 peers and TTL=3, the gossip reach at each hop (accounting for 25% clustering coefficient) is:

```
hop 1: 20 nodes
hop 2: 20 * 20 * (1 - 0.25) = 300 nodes
hop 3: 300 * 20 * (1 - 0.25) = 4,500 nodes
```

Cumulative reach: ∼4,820 nodes within 3 hops.

6.5 Rate Limiting and Gossip Hardening

The gossip layer implements per-source rate limiting to prevent amplification attacks. All hardening rules are enforced client-side—no relay modifications required:

- Kind 10055 (pact requests): 10 req/s per source pubkey.
- Kind 10057 (data requests): 50 req/s per source pubkey.

Rate limiting uses a sliding window counter per source. Excess requests are silently dropped. Combined with WoT-only forwarding, this bounds the gossip blast radius to the trust network while maintaining epidemic delivery within it.

Request deduplication uses an LRU cache of 10,000 request IDs. Duplicate requests are dropped silently, preventing gossip loops and reducing amplification.

6.6 Data Availability

The probability that all pact partners are simultaneously offline determines data unavailability:

Formula F-14 (all-pacts-offline probability):

$$P(\text{unavailable}) = (1 - u_{\text{full}})^{P \cdot f} \cdot (1 - u_{\text{light}})^{P \cdot (1-f)} \tag{10}$$

With default parameters ($P = 20$, $f = 0.25$, $u_{\text{full}} = 0.95$, $u_{\text{light}} = 0.30$):

$$P(\text{unavailable}) = (0.05)^5 \cdot (0.70)^{15} = 3.125 \times 10^{-7} \cdot 4.747 \times 10^{-3} = 1.48 \times 10^{-9} \tag{11}$$

This represents approximately one-in-a-billion chance of complete data unavailability at any instant—comparable to enterprise storage system reliability, achieved entirely through social graph redundancy.

7 Flow Control

7.1 The Capacity Problem

Van Renesse et al. [2] demonstrated that anti-entropy protocols have bounded capacity: under high update load, gossip messages cannot carry all required deltas, and update latency grows without bound. Our protocol faces an analogous constraint: each node's gossip bandwidth is finite, and the rate of storage pact requests, data requests, and event deliveries must be controlled.

7.2 Pact-Aware Priority

Rather than the Scuttlebutt reconciliation's approach of ordering deltas by version number, our protocol orders gossip forwarding by social proximity:

1. **Active pact partners**: forwarded immediately, no queuing.
2. **1-hop WoT**: forwarded with standard priority.
3. **2-hop WoT**: forwarded when capacity is available.
4. **Beyond WoT**: never forwarded.

This is analogous to scuttle-depth ordering [2]—the protocol prioritizes propagating information for the peers most relevant to the local node, rather than being "fair" across all participants.

7.3 Three-Phase Adoption

The protocol includes a built-in flow control mechanism through its adoption phases:

Phase	Pact Count	Behavior
Bootstrap	0–5	Publish to relays primarily; form pacts as available
Hybrid	5–15	Publish to both relays and peers; fetch from peers first
Sovereign	15+	Storage peers primary; relays optional accelerator

New users begin in Bootstrap phase with full relay dependence. As they form pacts, traffic gradually shifts from relays to peer mesh. This provides natural flow control: the rate at which a new node joins the gossip network is limited by the rate at which it forms pacts, preventing gossip overload from sudden influx.

7.4 Pact Renegotiation Jitter

When a user's activity changes and pact renegotiation is needed, the protocol introduces random jitter (0–48 hours) before broadcasting replacement requests. Standby pacts provide immediate failover during this delay. This prevents renegotiation storms—the gossip equivalent of TCP's synchronized loss recovery problem [5].

8 FIPS Integration: Beyond the Internet

8.1 Motivation

The protocol as described assumes IP connectivity between nodes. This creates a residual infrastructure dependency: the internet itself. FIPS (Free Internetworking Peering System) [6] eliminates this dependency by providing a self-organizing mesh network that operates natively over heterogeneous transports.

8.2 FIPS Architecture

FIPS implements three protocol layers:

Transport Layer: Delivers datagrams over arbitrary media—UDP/IP, Ethernet, Bluetooth Low Energy, serial links, radio modems, Tor circuits.

FIPS Mesh Protocol (FMP): Authenticates peers via Noise IK handshakes, builds a self-organizing spanning tree for coordinate-based routing, and propagates reachability via bloom filters.

FIPS Session Protocol (FSP): Provides end-to-end authenticated encryption via Noise XK handshakes. Sessions are bound to Nostr keypairs (npubs), not transport addresses, so they survive route changes and transport switching.

8.3 Identity Convergence

FIPS uses Nostr keypairs (secp256k1) as native node identities. This is the critical integration point: **a Gozzip user's root pubkey is also their FIPS network address**. No bridging, translation, or identity mapping is required. A storage pact partner is simultaneously a mesh routing peer.

The FIPS node_addr (a 16-byte SHA-256 hash of the pubkey) serves as the routing identifier in packet headers, while the npub serves as the application-layer address. Both are deterministically derived from the same keypair.

8.4 Transport Independence for Gossip

With FIPS as the transport layer, the gossip protocol operates identically regardless of the underlying medium:

Scenario	Transport Path	Gossip Behavior
Both nodes on internet	UDP/IP overlay	Standard gossip
One node on local mesh	BLE + WiFi	Gossip via mesh relay
Both nodes offline	BLE mesh (7 hops)	Local gossip, store-and-forward
Censored network	Tor transport	Gossip via onion routing
Remote/rural	Radio modem	Low-bandwidth gossip

8.5 Spanning Tree Meets Social Graph

FIPS builds a spanning tree for coordinate-based routing using distributed parent selection. Each node chooses a parent based on measured link quality (RTT, loss, jitter), creating a tree that reflects physical network topology.

The social graph's WoT structure overlays this physical topology. In many cases, WoT peers will also be FIPS mesh peers—a followed user's always-on full node is likely to be configured as a direct FIPS peer. Where social and physical topology diverge, FIPS's bloom filter discovery provides the bridge.

8.6 Offline-First Operation

FIPS enables a genuinely offline-first mode:

1. **BLE mesh transport** (Layer 0 in the delivery priority): nearby devices relay events up to 7 hops via Bluetooth Low Energy, encrypted with Noise Protocol XX handshakes.
2. **Store-and-forward queuing**: when no transport is available, events are queued locally (up to 1,000 events or 50MB) and auto-drain when any transport becomes available.
3. **Geohash discovery**: ephemeral subkeys (not linked to identity) with geohash tags enable nearby-device discovery without revealing identity.

8.7 The "Pub Server" Problem, Solved Differently

Scuttlebutt [7] identified the fundamental tension in peer-to-peer social networks: mobile devices go offline, but content must remain available. Scuttlebutt's solution was "pub servers"—always-on nodes that replicate data. But pub servers are relays by another name.

Our protocol addresses this through three mechanisms:

1. **Full nodes as pact partners**: The 25% of network participants that are always-on serve as full-history storage peers. Unlike pub servers, they have bilateral obligations enforced by challenge-response.
2. **Heterogeneous transport fallback**: When a mobile device goes offline on cellular, it may still be reachable via BLE mesh. FIPS routing finds the path.
3. **Standby pact promotion**: When a pact partner goes unreachable, a standby partner is immediately promoted—no renegotiation, no discovery delay.

9 Related Work

9.1 Anti-Entropy and Epidemic Protocols

The foundational work on epidemic algorithms for replicated database maintenance [8] established that updates spread in $O(\log N)$ rounds through random peer selection. Van Renesse et al. [2] extended this with the Scuttlebutt reconciliation mechanism and AIMD-based flow control. Our protocol adopts the epidemic spreading model but replaces random peer selection with WoT-weighted selection, sacrificing theoretical convergence speed for trust-bounded propagation.

9.2 Scuttlebutt / Secure Scuttlebutt

Secure Scuttlebutt (SSB) [7] proved the viability of gossip-based social networking with local-first storage and append-only feeds. Our protocol builds on SSB's core insight but differs in three ways:

1. **Nostr key model**: SSB uses ed25519 feeds tied to devices; we use secp256k1 keypairs with a delegation hierarchy that supports multi-device and key rotation.
2. **Bilateral pacts vs. unilateral replication**: SSB's pubs replicate unilaterally; our pacts enforce reciprocal obligation with proof of storage.
3. **WoT-bounded gossip**: SSB's gossip has no explicit trust boundary; our protocol restricts forwarding to 2-hop WoT distance.

9.3 Proof of Storage

The challenge-response proof of storage mechanism draws from Filecoin's Proof of Replication and Proof of Spacetime [9], and Arweave's Succinct Proofs of Random Access [10]. Our protocol uses simpler primitives because the threat model is different: pact partners are WoT peers with social incentive to maintain the relationship, not anonymous miners requiring cryptoeconomic guarantees.

9.4 FIPS and Mesh Networking

FIPS [6] provides the transport-agnostic mesh layer that our protocol requires for internet-independent operation. FIPS's use of Nostr keypairs for node identity creates a natural integration point. The spanning tree routing with bloom filter discovery is complementary to our WoT-based gossip routing: FIPS handles physical reachability while the gossip layer handles social relevance.

9.5 Local-First Software

The Ink & Switch research group's work on local-first software [11] articulated the principles our protocol implements: data ownership, offline operation, and collaboration without mandatory servers. Our contribution is extending these principles to a social networking context where the "collaborators" are an entire social graph, not a small team.

9.6 Infrastructure Benchmark: Nostr and Mastodon

The two most prominent decentralized social protocols—Nostr [1] and Mastodon/ActivityPub [12]—represent fundamentally different architectural choices. Nostr separates identity from infrastructure but depends on relay servers. Mastodon binds identity to infrastructure through domain-coupled accounts. Our protocol makes the social graph itself the infrastructure, while maintaining interoperability with both: Nostr events work natively (same key model, same relays), and data can be converted to ActivityPub format for Mastodon federation. The following tables provide a structured comparison.

Identity and Portability

	Nostr	**Mastodon**	**Gozzip**
Identity	secp256k1 keypair	`user@instance` (domain-bound)	secp256k1 + key hierarchy
Portability	Full—identity is the keypair	Partial—followers migrate; posts do not	Full—same Nostr key model; data convertible to ActivityPub/AT Protocol
Multi-device	Share private key or NIP-26	Native—server holds session	Root key delegates to device subkeys
Recovery	None—lose key, lose identity	Admin can reset password	Root key revokes subkeys; root loss is fatal

Data Storage and Ownership

	Nostr	**Mastodon**	**Gozzip**
Where data lives	Relay servers	Home instance (PostgreSQL)	User's device + ∼20 pact partners
Who controls	Relay operators	Instance admin	User + bilateral pact partners
Redundancy	Manual multi-relay publishing	None—single home instance	20 active + 3 standby pacts; $P(\text{all-offline}) \approx 10^{-9}$
If infra dies	Events lost unless duplicated	All posts from instance lost	Pact partners retain copies

Content Distribution

	Nostr	Mastodon	Gozzip
Mechanism	Client pulls from relays via WebSocket (REQ/EVENT)	Server pushes to followers' instances via HTTP POST to shared inbox	Tiered cascade: local pact (Tier 1) → cached endpoint (Tier 2) → WoT gossip (Tier 3) → relay fallback (Tier 4)
Discovery	Client connects to author's outbox relays (NIP-65)	Federated timeline, relay servers, hashtag trends	WoT-bounded gossip with blinded pubkeys; 2-hop propagation boundary
Read privacy	Relay sees all subscriptions and read patterns	Instance admin can see all activity; DMs stored unencrypted	Blinded pubkeys rotate daily; storage peers cannot identify the requester
Offline	Not supported	Not supported	BLE mesh (Tier 0) via FIPS; store-and-forward over radio, serial, Tor

Censorship Resistance

	Nostr	Mastodon	Gozzip
Censorship surface	Individual relays can drop events; mitigated by multi-relay publishing	Instance admin has unilateral suspend/delete power; domain blocks sever federation	No single point—data distributed across 20+ WoT peers
Content filtering	Per-relay policies; NIP-42 auth gating; proof-of-work spam deterrence (NIP-13)	Per-instance admin moderation; community-level domain blocklists	WoT filtering—gossip propagates only within 2-hop trust boundary; no global moderation authority
Account risk	Low (self-sovereign keypair) but relays can refuse to serve events	High—admin can suspend with no technical appeal mechanism	Very low—pact partners hold data under bilateral obligation

Scalability and Costs

	Nostr	Mastodon	Gozzip
Cost bearer	Relay operators (shifted to users via paid relays, ∼$5–20/mo)	Instance operators (media cache: 10–20 GB/day with relay subscriptions)	Distributed across participants—volume-balanced within 30% tolerance
Bandwidth	Variable; duplicate events across relays multiply client bandwidth	Federation delivery: one HTTP POST per remote instance with followers	Scales with pact count and event rate; bounded by volume balancing
Popular content	Each relay serves independently; no coordinated caching	Boosts replicate to each instance separately	Cascading read-caches: reads create replicas, scaling O(followers)
Infrastructure	∼1,000 relays (471 public, 191 restricted); concentration on high-traffic relays	∼26,000 instances; mastodon.social dominates with 348k+ accounts	25% Full nodes (always-on), 75% Light nodes (intermittent); no central infrastructure

Architectural Summary

Nostr's design cleanly separates identity from infrastructure, but relays remain the storage and distribution layer—recreating a dependency surface analogous to the platforms it replaces. The outbox model (NIP-65) and negentropy sync (NIP-77) improve relay coordination but do not eliminate relay dependence.

Mastodon bundles identity with infrastructure. The `user@instance` model means your account's survival depends on your instance operator's continued goodwill, funding, and uptime. FEP-ef61 (portable objects) and FEP-8b32 (object integrity proofs) aim to decouple identity from instances, but these remain proposals.

Our protocol addresses both failure modes by making the social graph the infrastructure layer. Storage pacts distribute data custody across WoT peers with cryptographic verification. The relay is demoted from gatekeeper to optional fallback. Critically, the protocol works with standard Nostr relays without any modifications—all Gozzip event kinds are valid Nostr events, and all protocol intelligence (gossip forwarding, blinded matching, WoT filtering, device resolution) lives in clients. Because events are self-authenticating (signed JSON), they can be converted to and from ActivityPub (Mastodon), AT Protocol (Bluesky), and other decentralized formats through bridge services, making the data portable across protocols.

The trade-off is protocol complexity: pact negotiation, challenge-response verification, WoT computation, and multi-tier retrieval are substantially more complex than Nostr's REQ/EVENT or Mastodon's HTTP POST delivery.

The honest gap: our protocol is validated analytically and by simulation. Nostr has ∼1,000 relays and real users. Mastodon has ∼26,000 instances and ∼1.2M monthly active users. The architectural claims require production-scale validation.

10 Conclusion

We have presented an open, censorship-resistant protocol for social media and messaging that returns data custody from server infrastructure to the social graph. The protocol inherits Nostr's proven primitives—secp256k1 identity, signed events, relay transport—and adds a storage and retrieval layer where users own their data. Because events are self-authenticating, they are portable: convertible to and from ActivityPub (Mastodon), AT Protocol (Bluesky), and other decentralized formats. The protocol works with standard Nostr relays without any modifications—all protocol intelligence lives in clients.

The protocol's design mirrors human social dynamics: reciprocal storage pacts parallel reciprocal friendships, WoT-filtered gossip parallels trust-based information sharing, and the Full/Light node distinction mirrors the inner-circle/outer-circle structure of human communities.

The tiered retrieval cascade ensures that the vast majority of reads are served from local pact storage, with relay fallback reserved for cold-start and beyond-graph content. As pacts mature, relay dependency decays naturally—the relay is demoted from gatekeeper to optional accelerator, useful during bootstrap and for distant authors, but not required for typical social reading patterns.

Integration with FIPS extends the protocol beyond internet dependence, enabling operation over mesh radio, Bluetooth, and other transports. The shared Nostr identity model eliminates bridging complexity: a user's social identity is simultaneously their network address.

The honest assessment: this architecture does not eliminate the need for always-on infrastructure. Full nodes—*Keepers*—(25% of the network) are the protocol's equivalent of reliable friends

who are always available. The difference is structural: these nodes operate under bilateral obligations enforced by challenge-response, not under unilateral control of a platform operator. The infrastructure exists, but it is owned by the social graph.

The hard problems remain. Graph bootstrap for new users requires initial relay dependence. The 75/25 Full/Light split must emerge organically through incentives, not be mandated. DM key rotation provides bounded forward secrecy but not perfect. These are engineering challenges, not architectural ones—the social graph is a viable foundation for decentralized infrastructure.

11 Implementation Roadmap

This section addresses a different question: how does a real network get from today's relay-dependent world to the sovereign architecture described above? The answer is not revolution but transition—a sequence of phases where each delivers concrete value before the next begins, and where the user experience changes only when the underlying infrastructure has already proven itself.

11.1 Design Philosophy: Transition, Not Revolution

The protocol must coexist with relays, not replace them on day one. Nostr's relay infrastructure works—and the protocol is designed so that standard Nostr relays require zero modifications. All Gozzip event kinds are valid Nostr events. Users have accounts, follow lists, and reading habits built on relays. Any viable deployment path must treat this as a starting point, not an obstacle.

The transition follows three principles:

1. **Invisible first, visible later.** The pact layer works silently underneath the existing client experience. Users should see zero behavioral change during Phase 1—no new UI, no configuration, no education required. Storage decentralization happens in the background before retrieval decentralization changes the read path.

2. **Each phase delivers value independently.** Phase 1 (storage redundancy) is valuable even if Phase 2 never ships. Phase 2 (fallback retrieval) is valuable even if Phase 3 never ships. No phase depends on the completion of a later phase for its value proposition to hold.

3. **Hardware trends are an ally.** What costs 10 MB/day and 5% battery today will cost nothing in three years. Mobile storage doubles every two years. 5G bandwidth is becoming ubiquitous. The protocol should be designed for the devices that will exist at deployment, not the devices that exist during simulation. Constraints that feel tight today will be invisible by the time Phase 3 deploys.

A critical framing: not everyone needs to be a full node. Full nodes don't need 100% uptime. The protocol's redundancy model means your data has *someone* holding it when you're offline—and with 20 pact partners, the probability that all of them are simultaneously unavailable is effectively zero (Section 6). The goal is not universal participation at maximum capacity; it is sufficient redundancy across realistic participation patterns.

11.2 Phase 1: Decentralize Storage

Goal: Every user's data exists in multiple places, not just relays.

The client works exactly like today—reads from relays, writes to relays. Nothing changes in the user-facing read or write path. In the background, the pact layer activates:

- **Pact formation begins** as users build their social graph. Mutual follows create WoT edges; the pact negotiation protocol (Section 5) finds volume-matched partners within the WoT boundary.
- **Pact partners silently replicate** each other's events. When Alice publishes a note to her relay, her 20 pact partners also receive and store it. This happens through the existing gossip channel—no additional user action required.
- **Challenge-response runs quietly.** Proof-of-storage verification (Section 5) operates on a background schedule. Users never see it. Failed challenges trigger partner replacement through the standby promotion mechanism.
- **No UX change. No user education needed. It just works.**

The value delivered is straightforward: if a relay goes down, deletes content, or gets censored, the data still exists on pact partners. This is *backup*, not *retrieval*—the relay remains primary for reads. But the single point of failure is eliminated.

Mobile cost during Phase 1 is modest. A light node storing 5 partners' recent events (30-day window) requires approximately 150 KB/day of sync traffic—less than loading a single web page. Storage obligation is bounded by the checkpoint window and volume balancing (Section 5).

Key metric: Data survival rate when relays fail. With mature pacts, the vast majority of reads should succeed from local pact storage without any relay involvement.

11.3 Phase 2: Opportunistic Retrieval

Goal: When relays fail, try pact partners before giving up.

The relay remains the primary read path. The change is what happens on failure:

- **Relay failure triggers pact fallback.** On relay timeout, 404, or offline status, the client silently tries pact partners of the target author before displaying an error. This is the Tier 1 → Tier 4 cascade (Section 6) with Tier 3 (gossip) activated as an intermediate step.
- **Users experience fewer failures.** Instead of "relay unreachable" errors, content loads anyway—from a pact partner who holds the author's data. The user sees "that post loaded" instead of an error message.
- **Clients learn which pact partners are reliable and fast.** Successful retrievals build a local routing table: for each author, which of their pact partners responded fastest? This is the CachedEndpoint tier (Tier 2) activating naturally—successful gossip and pact retrievals create endpoint hints for future reads.
- **This is the fallback phase—relay-first, pact-second.** The read path is: try relay → on failure, try known pact partners → on failure, gossip to WoT → on failure, display error.

Key metric: Failure rate reduction compared to relay-only operation.

11.4 Phase 3: WoT-Native Reads

Goal: Reads from close social contacts go to pact partners first, relays second.

This phase inverts the read priority for content within the user's Web of Trust:

- **Direct WoT reads (mutual follows) try pact partners before relays.** For authors who are likely pact partners—mutual follows with active storage agreements—the pact network is the primary read path. The relay becomes the fallback.
- **1-hop reads still try relays first.** Authors the user follows but who don't follow back are less likely to be pact partners, so the relay-first strategy remains appropriate for this tier.
- **The relay dependency decay curve kicks in.** Relay usage drops as pacts mature. This transition happens organically—no flag day, no coordinated switchover.
- **Trust-weighted gossip routing improves.** Forwarding decisions get smarter as the WoT graph stabilizes. Nodes learn which peers forward efficiently, which respond to gossip queries, and which are consistently offline. The gossip layer becomes a tuned routing mesh rather than a broadcast mechanism.

This is where the protocol's design goals—high instant delivery, near-total retrieval success—become the real-world experience. But the protocol doesn't need to start here. It earns this performance by building through Phases 1 and 2, where the pact network proves itself as a reliable storage and fallback layer before being trusted as the primary read path.

11.5 Phase 4: Transport Independence

Goal: The protocol works without the internet.

- **BLE mesh (Tier 0)** enables proximity exchange—two phones in the same room share events directly, relayed up to 7 hops via Bluetooth Low Energy through the FIPS transport layer (Section 8).
- **Store-and-forward queuing:** Events created offline are queued locally (up to 1,000 events or 50 MB) and drain automatically when any transport becomes available.
- **FIPS integration:** The same protocol runs over UDP, Ethernet, BLE, serial radio, and Tor. The gossip layer is transport-agnostic—it operates identically whether the underlying medium is fiber optic or a radio modem (Section 8).
- **Geohash discovery with ephemeral keys:** Nearby users discover each other using geohash-tagged ephemeral subkeys that are not linked to their identity, enabling proximity-based gossip without surveillance.

This phase serves specific audiences: activism under censorship, disaster response when infrastructure fails, off-grid communities, privacy maximalists who want zero internet dependency. Not every user reaches this phase. That's fine. The protocol is useful at every phase—Phase 4 is an extension of capability, not a requirement for value.

11.6 Mobile as First-Class Citizen

The intuition that mobile devices can't participate in a storage protocol deserves scrutiny. Modern smartphones have capabilities that exceed what the protocol requires:

Resource	Available	Protocol Requires	Margin
Storage	128–256 GB	<1 GB (power user)	100×+
Daily bandwidth	1–5 GB (WiFi)	4.1 MB/day (Light)	100×+
Background exec.	BGAppRefreshTask / WorkManager	Sync every 15–60 min	Native support
Battery	4,000–5,000 mAh	<5% daily for sync	≈ push notifications

The protocol's mobile architecture acknowledges that phones are intermittent participants, not always-on servers:

- **Light node by default.** The protocol is built for intermittent participation—challenge-response, pact obligations, and retrieval all account for variable availability.
- **Foreground mode:** Full gossip participation, real-time pact management, immediate challenge responses.
- **Background mode:** Periodic sync using OS-provided background task APIs (BGAppRefreshTask on iOS, WorkManager on Android). Challenge responses queue and resolve on the next background cycle. Heartbeats maintain pact partner awareness.
- **Sleeping mode:** Partners serve your data while you're offline. Challenges queue. You respond when you wake up. The standby pact mechanism (Section 5) ensures that temporary absence doesn't trigger pact drops—partners tolerate offline periods proportional to the light node uptime assumption.

Phone capabilities improve every year. What's a constraint today—background processing limits, storage tier pricing, cellular data caps—becomes trivial tomorrow. The protocol should be evaluated against the devices that will exist when Phase 3 deploys, not the devices available during Phase 1.

A "full node" doesn't need to be a phone. A desktop computer, a Raspberry Pi, or a $5/month VPS can serve as a full node—one technical friend running a full node serves 10–20 light nodes in their social circle as a high-uptime pact partner. This mirrors the social reality: every friend group has one person who runs the group server, hosts the shared drive, or keeps the archive. The protocol formalizes this role.

11.7 The Relay Doesn't Die

A critical framing error would be to read this roadmap as "relays go away." They don't. Relays remain valuable infrastructure with a changed role:

- **Discovery layer.** New users find content, authors, and communities through relays. The WoT graph doesn't help you find people you don't know yet—relays do.
- **Content curation.** Relay operators curate what they surface: filtering spam, promoting quality, organizing by topic. This is the relay's natural value proposition—editorial judgment, not data custody.
- **Performance accelerator.** For content from distant authors (beyond the 2-hop WoT boundary), relays provide faster access than multi-hop gossip. The relay is a CDN, not a database.
- **Bootstrap infrastructure.** New users start relay-dependent. That's by design, not a failure. The Bootstrap phase (Section 11) explicitly includes relay dependence, with pact formation gradually reducing it.

The protocol doesn't kill relays—it removes their monopoly on data custody. Relay operators become curators (choosing what to surface) rather than gatekeepers (deciding what exists). A relay that goes offline, changes its terms, or censors content no longer destroys the data it hosted—because that data exists on 20+ pact partners who have bilateral obligations to preserve it.

Critically, the relays involved are standard Nostr relays. No custom software, no vendor lock-in. Every Gozzip event kind is a valid Nostr event that any relay will accept, store, and serve. Relay operators who wish to optimize for Gozzip traffic can implement optional accelerators—oracle

resolution (caching device-to-root identity mappings), checkpoint delta delivery, gossip relay forwarding, NAT hole punching—but these are performance improvements, not requirements.

The transition is voluntary and invisible. Relay dependency drops naturally as pacts form. Users who never form pacts—because they're new, because they prefer relay-only operation, because their client doesn't implement the pact layer—continue to work exactly as they do today. The protocol adds capability without removing any existing functionality.

11.8 What We Don't Know Yet

This roadmap is a deployment plan for a protocol validated analytically, not by production experience. Several questions remain open:

- **Minimum viable network size for Phase 2.** At what user count does opportunistic pact retrieval deliver measurably better failure rates than relay-only? The effect should emerge early as pacts form, but real-world network density may differ from theoretical graph models.
- **Whether the 25% full node ratio emerges organically.** The simulation assumes 25% of nodes are always-on (Section 4). In practice, this ratio depends on whether enough users run desktop clients, VPS instances, or dedicated hardware. If the ratio falls significantly below 25%, data availability degrades. Whether social incentives (being a good pact partner earns you reliable storage from others) are sufficient to sustain this ratio is an empirical question.
- **Optimal challenge frequency on mobile.** Proof-of-storage challenges (Section 5) cost battery and bandwidth. Too frequent and mobile users drain resources; too infrequent and defection goes undetected. The current design uses exponential moving average scoring with $\alpha = 0.95$, giving a 30-day effective window, but the optimal challenge interval for mobile devices specifically has not been empirically determined.
- **How CachedEndpoint tier performs in production.** Endpoint caching requires successful prior retrievals to populate the cache. In a real deployment with persistent storage across sessions, this tier should activate naturally—but its actual contribution to read performance is unknown.
- **Better retrieval algorithms.** The current retrieval cascade is a simple priority waterfall (Section 6). More sophisticated approaches—predictive caching based on follow-graph activity patterns, pre-emptive gossip for likely-to-be-read content, ML-driven routing optimization—may exist by the time Phase 3 deploys. The architecture should accommodate algorithm improvements without protocol changes, and the four-tier structure is designed to be extensible.
- **Economic sustainability of full nodes.** Full nodes bear disproportionate storage and bandwidth costs. Whether the social incentive (reliable storage reciprocity) is sufficient, or whether economic incentives (micropayments, premium services) are needed, remains to be determined.

These gaps are not architectural blockers. The protocol's core mechanisms—pact formation, challenge-response verification, WoT-filtered gossip, tiered retrieval—are analytically sound. The unknowns are deployment parameters that require production data to resolve—engineering challenges, not architectural ones.

References

[1] Nostr Protocol. "Nostr Implementation Possibilities." https://github.com/nostr-protocol/nips

[2] R. van Renesse, D. Dumitriu, V. Gough, C. Thomas. "Efficient Reconciliation and Flow Control for Anti-Entropy Protocols." LADIS Workshop, 2008.

[3] R. Dunbar. "Neocortex size as a constraint on group size in primates." *Journal of Human Evolution*, 22(6):469–493, 1992.

[4] M. Granovetter. "The Strength of Weak Ties." *American Journal of Sociology*, 78(6):1360–1380, 1973.

[5] V. Jacobson. "Congestion Avoidance and Control." ACM SIGCOMM, 1988.

[6] J. Corgan. "FIPS: Free Internetworking Peering System." https://github.com/jmcorgan/fips

[7] D. Tarr, E. Lavoie, A. Chen, J. Robinson. "Secure Scuttlebutt: An Identity-Centric Protocol for Subjective and Decentralized Applications." ACM PLDI, 2019.

[8] A. Demers, D. Greene, C. Hauser, et al. "Epidemic Algorithms for Replicated Database Maintenance." ACM PODC, 1987.

[9] Protocol Labs. "Filecoin: A Decentralized Storage Network." 2017.

[10] S. Williams, V. Diiorio, S. Barski. "Arweave: A Protocol for Economically Sustainable Information Permanence." 2019.

[11] M. Kleppmann, A. Wiggins, P. van Hardenberg, M. McGranaghan. "Local-First Software: You Own Your Data, in Spite of the Cloud." Onward!, 2019.

[12] C. Webber, J. Tallon. "ActivityPub." W3C Recommendation, 2018. https://www.w3.org/TR/activitypub/

[13] D. Watts, S. Strogatz. "Collective dynamics of 'small-world' networks." *Nature*, 393:440–442, 1998.

[14] A.-L. Barabási, R. Albert. "Emergence of Scaling in Random Networks." *Science*, 286(5439):509–512, 1999.

[15] R. Albert, H. Jeong, A.-L. Barabási. "Error and attack tolerance of complex networks." *Nature*, 406:378–382, 2000.

[16] R. Cohen, K. Erez, D. ben-Avraham, S. Havlin. "Resilience of the Internet to random breakdowns." *Physical Review Letters*, 85(21):4626–4628, 2000.

[17] R. Pastor-Satorras, A. Vespignani. "Epidemic spreading in scale-free networks." *Physical Review Letters*, 86(14):3200–3203, 2001.

[18] C. Castellano, R. Pastor-Satorras. "Thresholds for epidemic spreading in networks." *Physical Review Letters*, 105(21):218701, 2010.

[19] A. Rapoport. "Spread of information through a population with socio-structural bias: I. Assumption of transitivity." *Bulletin of Mathematical Biophysics*, 15(4):523–533, 1953.

[20] M. Girvan, M. Newman. "Community structure in social and biological networks." *Proceedings of the National Academy of Sciences*, 99(12):7821–7826, 2002.

[21] K. Menger. "Zur allgemeinen Kurventheorie." *Fundamenta Mathematicae*, 10:96–115, 1927.